

Regularization in Neural Network

Mathematical Foundations and Examples

Moin Mostakim

What is Regularization?

Definition

Technique to prevent overfitting by adding penalty to loss function

Core Idea

$$J_{\text{reg}}(\theta) = \underbrace{J(\theta)}_{\text{Original Loss}} + \lambda \cdot \underbrace{R(\theta)}_{\text{Penalty Term}}$$

- $J(\theta)$: Original loss function (e.g., MSE, Cross-Entropy)
- $R(\theta)$: Regularization term
- $\lambda \geq 0$: Regularization parameter (controls strength)

Why Regularize?

The Overfitting Problem

Overfitting:

- Model fits training data too closely
- Captures noise instead of pattern
- Poor generalization to new data

Regularization Helps:

- 1 Reduces model complexity
- 2 Improves generalization
- 3 Controls variance
- 4 Handles multicollinearity

Goal: Find the sweet spot between underfitting and overfitting

Common Regularization Methods

Mathematical Forms

1 L_2 Regularization (Ridge)

$$R(\theta) = \frac{1}{2} \|\theta\|_2^2 = \frac{1}{2} \sum_{j=1}^n \theta_j^2$$

2 L_1 Regularization (Lasso)

$$R(\theta) = \|\theta\|_1 = \sum_{j=1}^n |\theta_j|$$

3 **Elastic Net** ($L_1 + L_2$)

$$R(\theta) = \alpha \|\theta\|_1 + (1 - \alpha) \frac{1}{2} \|\theta\|_2^2$$

Example 1: Ridge Regression (L_2)

Closed-Form Solution

Optimization Problem

$$\min_{\theta} \left[\frac{1}{2m} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2 + \frac{\lambda}{2} \sum_{j=1}^n \theta_j^2 \right]$$

Closed-Form Solution

$$\hat{\theta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

Key Property

$X^T X + \lambda I$ is always invertible for $\lambda > 0$

Derivation of Ridge Solution

Setting Gradient to Zero

$$\begin{aligned}\nabla_{\theta} J_{\text{ridge}}(\theta) &= 0 \\ \frac{\partial}{\partial \theta} \left[\frac{1}{2} \|y - X\theta\|^2 + \frac{\lambda}{2} \|\theta\|^2 \right] &= 0 \\ -X^T(y - X\theta) + \lambda\theta &= 0 \\ (X^T X + \lambda I)\theta &= X^T y \\ \theta &= (X^T X + \lambda I)^{-1} X^T y\end{aligned}$$

$\lambda > 0$ ensures matrix invertibility even when $X^T X$ is singular

Example 2: Lasso Regression (L_1)

No Closed-Form Solution

Optimization Problem

$$\min_{\theta} \left[\frac{1}{2m} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2 + \lambda \sum_{j=1}^n |\theta_j| \right]$$

Non-differentiable at zero!

The $|\theta_j|$ term is not differentiable at $\theta_j = 0$

Solution Methods

- Coordinate descent
- Subgradient methods
- Proximal algorithms
- Least Angle Regression (LARS)

Geometric Interpretation

Constrained Formulation

Equivalent Constrained Form

$$\min_{\theta} J(\theta) \quad \text{s.t.} \quad R(\theta) \leq t$$

L_2 (**Ridge**):

$$\|\theta\|_2^2 \leq t$$

Circular/spherical constraint region

L_1 (**Lasso**):

$$\|\theta\|_1 \leq t$$

Diamond-shaped constraint region

Gradient Descent Updates

Comparing Different Regularizations

Method	Update Rule
No Regularization	$\theta_j^{(t+1)} = \theta_j^{(t)} - \alpha \frac{\partial J(\theta^{(t)})}{\partial \theta_j}$
L_2 Regularization	$\theta_j^{(t+1)} = \theta_j^{(t)} - \alpha \left[\frac{\partial J(\theta^{(t)})}{\partial \theta_j} + \frac{\lambda}{m} \theta_j^{(t)} \right]$
L_1 Regularization	$\theta_j^{(t+1)} = \theta_j^{(t)} - \alpha \left[\frac{\partial J(\theta^{(t)})}{\partial \theta_j} + \frac{\lambda}{m} \text{sign}(\theta_j^{(t)}) \right]$

L_1 's Sign Function

$$\text{sign}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Example 3: Logistic Regression with L

Classification Example

Original Loss (Cross-Entropy)

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right]$$

where $h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$

Regularized Version

$$J_{\text{reg}}(\theta) = J(\theta) + \frac{\lambda}{2m} \sum_{j=1}^n \theta_j^2$$

Gradient Update

$$\theta_j := \theta_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} + \frac{\lambda}{m} \theta_j \right]$$

Regularization Path: L_1 vs L_2

Parameter Shrinkage as λ Increases

L_2 (Ridge):

$$\theta_j(\lambda) = \frac{\theta_j(0)}{1 + \lambda}$$

Continuous shrinkage toward zero

L_1 (Lasso):

$$\theta_j(\lambda) = \text{sign}(\theta_j(0)) \cdot \max(|\theta_j(0)| - \lambda, 0)$$

Thresholding to exact zero!

Elastic Net Example

Best of Both Worlds

Penalty Term

$$R(\theta) = \lambda \left[\frac{1-\alpha}{2} \|\theta\|_2^2 + \alpha \|\theta\|_1 \right]$$

Optimization Problem

$$\min_{\theta} \frac{1}{2m} \|y - X\theta\|^2 + \lambda \left[\frac{1-\alpha}{2} \|\theta\|_2^2 + \alpha \|\theta\|_1 \right]$$

Special Cases

- $\alpha = 0$: Pure Ridge regression
- $\alpha = 1$: Pure Lasso regression
- $0 < \alpha < 1$: Balance between sparsity and shrinkage

Choosing λ : Cross-Validation

Finding the Optimal Regularization Strength

k-Fold Cross-Validation Procedure

- 1 Split data into k folds
- 2 For each λ value:
 - Train on $k-1$ folds
 - Validate on held-out fold
- 3 Choose λ with best validation performance

Bias-Variance Tradeoff

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

- **Small λ** : Low bias, high variance (overfitting)
- **Large λ** : High bias, low variance (underfitting)
- **Optimal λ** : Balanced bias-variance

Practical Considerations

Important Implementation Details

Feature Scaling is Crucial!

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

Regularization is **sensitive to scale** - always standardize!

Intercept Term

Typically **not regularized** (exclude θ_0 from penalty)

Python Implementation

```
Scale features scaler = StandardScaler() X_scaled = scaler.fit_transform(X)
Fit with regularization model = Ridge(alpha=1.0) alpha = model.fit(X_scaled, y)
```

Summary Table

Comparison of Regularization Methods

Aspect	L_2 (Ridge)	L_1 (Lasso)	Elastic Net
Penalty	$\ \theta\ _2^2$	$\ \theta\ _1$	$\alpha\ \theta\ _1 + (1 - \alpha)\ \theta\ _2^2$
Solution	Analytic	Iterative	Iterative
Sparsity	No	Yes	Controlled
Feature Selection	No	Yes	Yes
Correlated Features	Groups together	Selects one	Compromise
Computation	Fast	Moderate	Moderate

Key Takeaways

- 1 **Regularization controls model complexity** to prevent overfitting
- 2 L_1 **promotes sparsity** - useful for feature selection
- 3 L_2 **stabilizes solutions** - handles multicollinearity
- 4 λ **balances bias-variance tradeoff** - choose via cross-validation
- 5 **Always standardize features** before applying regularization
- 6 **Elastic Net** combines benefits of L_1 and L_2

$$J_{\text{reg}}(\theta) = J(\theta) + \lambda R(\theta)$$

Intuition: Noise Injection

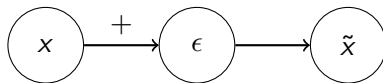
Basic Idea

Adding small random noise to inputs during training:

$$\tilde{x}^{(i)} = x^{(i)} + \epsilon^{(i)}, \quad \epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2 I)$$

Effect

Forces model to be robust to small input variations



Theorem Statement

Key Result

For linear regression with sum of squares loss, adding isotropic Gaussian noise to inputs is equivalent to L_2 regularization.

$$\text{Noise Injection} \equiv L_2 \text{ Regularization}$$

Precise Statement

Let $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$. Then:

$$\mathbb{E}_{\epsilon}[J(\theta; x + \epsilon, y)] = J(\theta; x, y) + \sigma^2 \|\theta\|_2^2$$

Original Data:

$$\{(x^{(i)}, y^{(i)})\}_{i=1}^m$$

$$x^{(i)} \in \mathbb{R}^n$$

Noisy Inputs:

$$\tilde{x}^{(i)} = x^{(i)} + \epsilon^{(i)}$$

$$\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2 I)$$

Loss Function:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2$$

Expected Loss:

$$J_{\text{noise}}(\theta) = \frac{1}{2m} \sum_{i=1}^m \mathbb{E}_{\epsilon} [(\theta^T \tilde{x}^{(i)} - y^{(i)})^2]$$

Derivation: Step 1 - Expand the Square

For each sample i :

$$\begin{aligned}\mathbb{E}_{\epsilon} \left[(\theta^T (x^{(i)} + \epsilon) - y^{(i)})^2 \right] \\&= \mathbb{E}_{\epsilon} \left[(\theta^T x^{(i)} - y^{(i)} + \theta^T \epsilon)^2 \right] \\&= \mathbb{E}_{\epsilon} \left[(\theta^T x^{(i)} - y^{(i)})^2 + 2(\theta^T x^{(i)} - y^{(i)})(\theta^T \epsilon) + (\theta^T \epsilon)^2 \right]\end{aligned}$$

Three Terms:

- 1 Original squared error
- 2 Cross term (linear in ϵ)
- 3 Quadratic term in ϵ

Derivation: Step 2 - First Two Terms

First Term (No Expectation Needed)

$$\mathbb{E}[(\theta^T x^{(i)} - y^{(i)})^2] = (\theta^T x^{(i)} - y^{(i)})^2$$

Second Term (Linear in ϵ)

$$\mathbb{E}[2(\theta^T x^{(i)} - y^{(i)})(\theta^T \epsilon)] = 2(\theta^T x^{(i)} - y^{(i)})\theta^T \mathbb{E}[\epsilon]$$

Since $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$:

$$\mathbb{E}[\epsilon] = 0 \quad \Rightarrow \quad \mathbb{E}[\text{cross term}] = 0$$

Derivation: Step 3 - Quadratic Term

Compute $\mathbb{E}[(\theta^T \epsilon)^2]$

$$(\theta^T \epsilon)^2 = \theta^T \epsilon \epsilon^T \theta$$

Take Expectation

$$\mathbb{E}[(\theta^T \epsilon)^2] = \theta^T \mathbb{E}[\epsilon \epsilon^T] \theta$$

For $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$

$$\mathbb{E}[\epsilon \epsilon^T] = \sigma^2 I$$

Therefore:

$$\mathbb{E}[(\theta^T \epsilon)^2] = \sigma^2 \theta^T \theta = \sigma^2 \|\theta\|_2^2$$

Derivation: Step 4 - Combine All Terms

Expected Loss for Sample i

$$\begin{aligned}\mathbb{E}_{\epsilon} \left[(\theta^T x^{(i)} + \epsilon - y^{(i)})^2 \right] \\&= (\theta^T x^{(i)} - y^{(i)})^2 + 0 + \sigma^2 \|\theta\|_2^2 \\&= (\theta^T x^{(i)} - y^{(i)})^2 + \sigma^2 \|\theta\|_2^2\end{aligned}$$

Total Loss Over All Samples

$$J_{\text{noise}}(\theta) = \frac{1}{2m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2 + \frac{\sigma^2}{2} \|\theta\|_2^2$$

Derivation: Step 5 - Compare with L_2 Regularization

Standard L_2 Regularization

$$J_{L_2}(\theta) = \frac{1}{2m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2 + \frac{\lambda}{2} \|\theta\|_2^2$$

Loss with Noise

$$J_{\text{noise}}(\theta) = \frac{1}{2m} \sum_{i=1}^m (\theta^T x^{(i)} - y^{(i)})^2 + \frac{\sigma^2}{2} \|\theta\|_2^2$$

Equivalence

$$\lambda = \sigma^2$$

Conclusion: Adding Gaussian noise with variance σ^2 is equivalent to L_2 regularization with $\lambda = \sigma^2$.

Why Does This Work?

Noise injection creates "virtual" training examples:

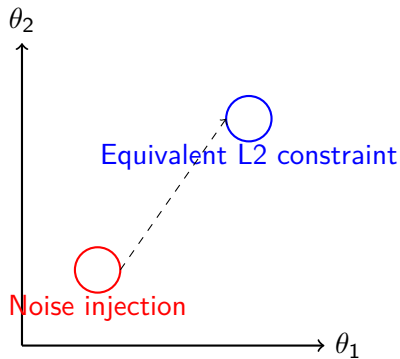
$$\tilde{x} = x + \epsilon$$

where ϵ is small random perturbation

Effect on Model

- Penalizes large parameter values
- Encourages smoother decision boundaries
- Improves generalization
- Equivalent to training on augmented dataset

Geometric Interpretation



Exact Equivalence:

- Holds for linear regression with MSE
- Linear models only
- Isotropic Gaussian noise

Neural Networks:

- Approximately equivalent
- L_2 regularization on first layer weights
- Not exact for deep networks

Non-Isotropic Noise: If

$$\epsilon \sim \mathcal{N}(0, \Sigma):$$

$$\mathbb{E}[(\theta^T \epsilon)^2] = \theta^T \Sigma \theta$$

Equivalent to Mahalanobis regularization:

$$J = \text{MSE} + \theta^T \Sigma \theta$$

Other Loss Functions:

- Not equivalent for cross-entropy
- Different regularization effects

Summary

Key Result

For linear regression with MSE loss:

$$\mathbb{E}_{\epsilon}[J(\theta; x + \epsilon, y)] = J(\theta; x, y) + \sigma^2 \|\theta\|_2^2$$

Interpretations

- 1 **Noise injection** $\approx$ **Data augmentation**
- 2 Forces model to be **smooth** and **robust**
- 3 Equivalent to **penalizing large weights**

Practical Insight

Choose σ (noise std) based on desired regularization strength:

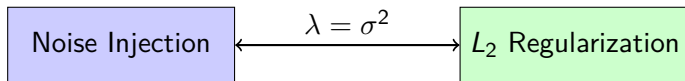
$$\lambda = \sigma^2$$

Final Visualization

$$\underbrace{\text{Train with } (x + \epsilon)}_{\text{Noise Injection, } \epsilon \sim \mathcal{N}(0, \sigma^2 I)}$$

$\Downarrow$ Equivalent to

$$\underbrace{\text{Train with } x + \lambda \|\theta\|_2^2}_{L_2 \text{ Regularization, } \lambda = \sigma^2}$$



Both improve generalization through implicit weight decay